

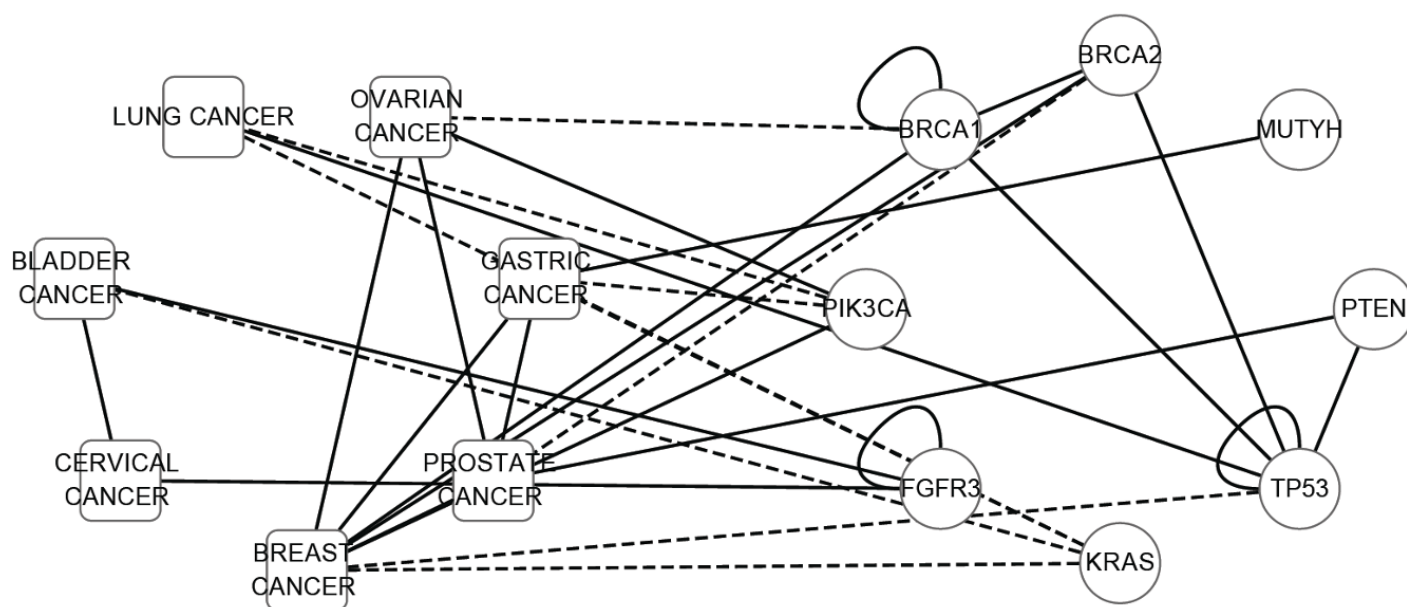
基于MindSpore的致病基因预测

实验目的

1. 使用MindSpore实现GCN模型。
2. 使用GCN模型进行致病基因预测，了解机器学习模型解决排序问题。

实验数据

1. **phenotype_network.mat**: 疾病与疾病间的相似度关系矩阵，维度为5080*5081，其中第一列为疾病的OMIM ID。
2. **g_p_network.mat**: 基因与疾病之间的关联关系矩阵，维度为8919*5080。
3. **ppi_network.mat**: 基因与基因之间的相似关系矩阵，维度为8919*8919。



左侧是**疾病表型网络**（disease phenotype network），右侧是**蛋白质交互网络**（protein-protein interaction network，PPI网络，因为蛋白质可使用基因去编码，所以PPI网络也可被认为是基因网络），中间是**疾病表型和基因网络**（g_p_network），是疾病表型和基因的二部图（bipartite network）。

实验步骤

1. 装载数据

该部分将三个网络数据进行加载，分别存放在 `phenotype_network`、`ppi_network` 和 `g_p_network` 中。其中疾病相似度网络 `phenotype_network` 的第一列为疾病表型的OMIM ID，

下图是该数据集的截图。

	0	1	2	3	4	5	6	7
		DiseaseID						
0	100050	1	0.0890344	0.361651	0.266263	0.368639	0.197142	0.0678806
1	100070	0.0890344	1	0.0873663	0.0267854	0.148069	0.233403	0.125448
2	100100	0.361651	0.0873663	1	0.0423538	0.269991	0.112267	0.0739974
3	100200	0.266263	0.0267854	0.0423538	1	0.215814	0.189442	0
4	100300	0.368639	0.148069	0.269991	0.215814	1	0.232661	0.0748263
5	100600	0.197142	0.233403	0.112267	0.189442	0.232661	1	0.0776241
6	100675	0.0678806	0.125448	0.0739974	0	0.0748263	0.0776241	1
7	100700	0.203997	0.0727986	0.2932	0.108829	0.243982	0.145725	0
8	100800	0.238039	0.0895563	0.170992	0.0935921	0.297043	0.214755	0
9	100820	0.210782	0.0539512	0.0852404	0.210522	0.107865	0.162785	0
10	101000	0.15004	0.15665	0.120332	0.137005	0.227114	0.206732	0.0935284
11	101120	0.368094	0.0315105	0.19298	0.120919	0.342189	0.163983	0

使用 `scipy.io.loadmat()` 装载上述3个数据集。每个mat文件只存了一个矩阵，矩阵名与mat文件名相同。

代码块

```
1 import numpy as np
2 import scipy.io
3
4 # 加载ppi_network, 该网络储存了基因与基因之间的相似关系。基因总数为8919, 故该网络维度为
  8919*8919。
5 ppi = loadmat("ppi_network.mat")
6 ppi_network = ppi["ppi_network"]
7 gene_name = ppi["gene_name"]
8
9 # 加载phenotype_network, 该网络储存了疾病与疾病间的相似度关系。疾病总数为5080。注意第一
  列为疾病ID, 故该网络维度为5080*5081
10 phenotype = loadmat("phenotype_network.mat")
11 phenotype_network = phenotype["phenotype_network"]
12 phenotype_name = phenotype["phenotype_name"]
13
14 # 加载g_p_network, 该网络储存了基因与疾病间的关联关系, 若第i个基因与第j个疾病有关, 则该
  位置元素为1, 无关为0。维度为8919*5080。
15 g_p_network = loadmat("g_p_network.mat")["g_p_network"]
```

2. 获取致病基因集合

2.1 返回目标疾病的最相似的5个疾病表型

这段代码通过给定的一个疾病ID（`DiseaseID`）和一个疾病表型网络（`phenotype_network`），找出与该疾病最相似的5个疾病的ID及其对应的疾病描述。

该部分定义了相似疾病查找函数 `getSimilarDiseases(DiseaseID, phenotype_network)`，其中参数 `DiseaseID` 为待预测致病基因对应的疾病ID。该ID可通过查看 `phenotype_network` 确定。

该函数接收疾病ID和疾病相似度网络后，返回两个结果：一个是与待预测疾病在表型上最相关的5个疾病的ID（存储在 `similarDiseaseID` 中）和这些相似疾病在 `phenotype_network` 中对应的行索引位置（存储在 `similarDiseasePos` 中）。

函数内部首先通过 `getIndexDisease` 查找 `DiseaseID` 在相似度网络中的行号，并提取该行与其他疾病的相似度向量；随后通过排序获取相似度最高的5个疾病的位置，并返回它们的ID和索引。

代码块

```
1  def getIndexDisease(DiseaseID):
2      m = phenotype_network.shape[0]
3      line_num = 0
4      for i in range(m):
5          if phenotype_network[i, 0] == DiseaseID:
6              line_num = i
7              break
8      return line_num
9
10 def getSimilarDiseases(DiseaseID, phenotype_network):
11     line_num = getIndexDisease(DiseaseID)
12
13     similarity_vec = phenotype_network[line_num, 1:]
14     # 对similarity_vec排序，不返回排序结果，返回排序值的index
15     pos = np.argsort(similarity_vec)
16     # 从倒数第5个元素开始取到最后。
17     similarDiseasePos = pos[-5:]
18
19     similarDiseaseID = phenotype_network[similarDiseasePos, 0]
20
21     return similarDiseaseID, similarDiseasePos
```

如果查询输入是114480（BREAST CANCER），那么最相似的5个疾病表型显示如下。其中，会显示选择的疾病ID及其对应描述，相似病理的5个疾病ID，最后会显示对应的疾病描述。

选择的疾病ID为: 114480,其对应描述为:['BREAST CANCER']
相似病理的ID为:
[155720. 120435. 113705. 176807. 114480.]
对应的疾病描述依次为:
MELANOMA, UVEAL
LYNCH SYNDROME I
BREAST CANCER 1 GENE; BRCA1
PROSTATE CANCER
BREAST CANCER

2.2 获取致病基因集合

这段代码基于目标疾病及其相似疾病，构建了一个疾病集合，并从基因-疾病网络中找出所有与该疾病集合对应的致病基因的编号与基因名称。

这部分首先构建疾病集合索引 `DiseaseSetPos`：其中 `similarDiseasePos` 表示与目标疾病最相似的5个疾病的位置；`getIndexDisease(DiseaseID)` 获取目标疾病自身在 `phenotype_network` 中的位置；将这两个部分合并为一个完整的疾病集合索引 `DiseaseSetPos`。

其次，查找与疾病集合相对应的基因索引 `GeneSetPos`：遍历每个疾病位置 `DiseasePos`；在 `g_p_network` 中查找哪些基因（行）与该疾病（列）存在关联（即值为1）；将这些基因的行索引添加到 `GeneSetPos` 中。

注意去重处理：对 `GeneSetPos` 进行去重，避免重复记录相同的致病基因。

最后，输出致病基因集合及其名称描述：打印最终的致病基因编号列表；遍历每个基因位置，结合 `gene_name` 列表输出基因名称。

代码块

```
1  # 获取疾病集合的下标
2  similarDiseasePos_List = similarDiseasePos.tolist()
3  DiseaseSetPos = similarDiseasePos_List + [getIndexDisease(DiseaseID)]
4
5  # 获取对应的基因ID
6  GeneSetPos = []
7  Gene_nums = ppi_network.shape[0]
8  for DiseasePos in DiseaseSetPos:
9      for i in range(Gene_nums):
10         if g_p_network[i,DiseasePos] == 1:
11             GeneSetPos.append(i)
12
13  # 去重
14  GeneSetPos = list(set(GeneSetPos))
```

```
15 print("致病基因")
16 print(GeneSetPos)
17 # 输出致病基因描述
18 print("致病基因集合描述：")
19 for Gene_Pos in GeneSetPos:
20     print(f"基因 {Gene_Pos}: {gene_name[Gene_Pos][0][0]}")
```

根据获得的相似病理疾病，获得对应基因集合显示如下。其中，会显示致病基因编号与基因名称。

致病基因

[1376, 1634, 101, 5447, 1479, 1385, 6733, 3062, 1370]

致病基因集合描述：

基因 1376: RAD51

基因 1634: PTEN

基因 101: BRCA1

基因 5447: RNASEL

基因 1479: NBN

基因 1385: BRCA2

基因 6733: ELAC2

基因 3062: PIK3CA

基因 1370: MSH2

3. 构造基于GCN的致病基因预测模型

什么是图卷积网络（GCN）

图卷积网络（Graph Convolutional Network, GCN）是一种用于处理图结构数据的神经网络，能够对图中的节点、边以及其关系进行建模。GCN被广泛用于社交网络分析、生物信息学（如蛋白质-蛋白质相互作用网络）、推荐系统等领域。

图结构的一个特点是数据之间的连接不规则，传统的卷积神经网络（CNN）无法直接处理。而GCN的目标是将邻接信息与节点特征共同编码为低维表示（embedding）。

GCN 的基本原理

GCN的核心思想是：每一层GCN都通过聚合节点邻居的信息来更新节点的特征表示。

GCN 的数学公式如下：

$$H^{(l+1)} = \sigma \left(\hat{A} H^{(l)} W^{(l)} \right)$$

- $H^{(l)}$ ：第 l 层节点特征， $H^{(0)} = X$
- $\hat{A}$ ：归一化处理后的邻接矩阵

- - $W^{(l)}$: 第 l 层可训练的权重矩阵
- - σ : 非线性激活函数 (如 ReLU)

3.1 定义GCN模型

这段代码使用MindSpore定义一个GCN模型，并创建。

该部分引入 MindSpore 深度学习框架及其基础模块。首先使用 MindSpore 框架定义的一个神经网络层，继承自 `nn.Cell`。 `GCNLayer` 是图神经网络 (GNN) 中的基本组成单元，用于对图中节点的特征进行更新。

首先初始化函数：定义参数。 `in_channels` : 输入特征的维度； `out_channels` : 输出特征的维度；使用 `nn.Dense(..., has_bias=False)` 定义了一个无偏置项的线性变换权重矩阵

$$W \in \mathbb{R}^{\text{in_channels} \times \text{out_channels}}$$

前向传播逻辑 `construct(self, X, A)`。 `X` : 节点特征矩阵； `A` : 邻接矩阵。 `support = self.weight(X)` : 先对节点特征进行线性变换 XW ； `output = ops.matmul(A, support)` : 再通过邻接矩阵进行邻居信息聚合，即 $A \cdot XW$ 。

代码块

```
1  import mindspore as ms
2  from mindspore import nn, ops, Tensor
3
4  # 定义 GCN 层
5  class GCNLayer(nn.Cell):
6      def __init__(self, in_channels, out_channels):
7          super(GCNLayer, self).__init__()
8          self.weight = nn.Dense(in_channels, out_channels, has_bias=False)
9
10     def construct(self, X, A):
11         support = self.weight(X)
12         output = ops.matmul(A, support)
13         return output
14
```

这部分定义了一个两层的图卷积神经网络 (GCN) 模型 `GCNModel`，用于对图中每个节点生成嵌入表示 (embedding)。

模型初始化 `__init__` :

- `GCNLayer` : 自定义的图卷积层，接收节点特征 `X` 和邻接矩阵 `A`，输出特征表示。
- 第一层：输入维度为 `feature_dim` (通常是节点特征维度)，输出为 `hidden_dim` (隐藏维度)
- 第二层：输入为 `hidden_dim`，输出为最终嵌入维度 `out_dim`

- 中间加入 ReLU 激活函数和 Dropout 层，防止过拟合(ReLU属于 非线性激活函数，用于在每层神经网络后增加非线性特征表达能力。Dropout是一种 随机失活的正则化技术，用于缓解过拟合，增强模型的泛化能力。它并不是激活函数，而是训练策略的一部分。)

前向传播 `construct` :

1. 单位矩阵作为初始特征:

- 我们没有节点本身的特征，所以用单位矩阵当作节点的 one-hot 特征（即每个节点的独立标识）
- 这样每个节点的初始输入是一个唯一的 one-hot 向量，维度为 `[num_nodes, num_nodes]`

2. 邻接矩阵转换为张量:

- 将邻接矩阵从 NumPy 转为 float32 并封装成张量，以便后续图卷积中使用

3. 第一层图卷积 + ReLU + Dropout

- 输入特征：one-hot 的单位矩阵
- 第一层 GCNLayer 根据图结构 `A_tensor` 汇聚邻居信息并映射到 `hidden_dim`
- ReLU 增加非线性表达能力
- Dropout 随机丢弃部分神经元，防止过拟合

4. 第二层图卷积（输出层）

- 第二层 GCN 继续根据图结构对第一层的输出特征进行卷积
- 输出维度为 `out_dim`，即我们最终想得到的节点嵌入表示

层	操作	输入维度 → 输出维度	说明
输入特征	单位矩阵 (one-hot)	[num_nodes, num_nodes]	没有原始特征，用单位矩阵表示节点身份
第一层 GCN	GCN + ReLU + Dropout	[num_nodes, feature_dim] → [num_nodes, hidden_dim]	提取初步图结构信息
第二层 GCN	GCN	[num_nodes, hidden_dim] → [num_nodes, out_dim]	得到最终嵌入表示

代码块

```
1  # 定义升级版 GCN 模型
2  class GCNModel(nn.Cell):
3      def __init__(self, hidden_dim=128, out_dim=64, dropout_rate=0.5,
4          feature_dim=64):
5          super(GCNModel, self).__init__()
6          self.gc1 = GCNLayer(feature_dim, hidden_dim)
7          self.gc2 = GCNLayer(hidden_dim, out_dim)
8          self.relu = ops.ReLU()
9          self.dropout = nn.Dropout(keep_prob=1 - dropout_rate)
```



```

10     def construct(self, A):
11         num_nodes = A.shape[0]
12
13         # 生成单位矩阵特征
14         X = Tensor(np.eye(num_nodes, dtype=np.float32)) # [num_nodes,
num_nodes]
15         A_tensor = Tensor(A.astype(np.float32))
16         '''code here'''
17         # 第一层h1
18
19         # 第二层h2
20         ''''''
21         return h2
22
23     # 加载预训练模型
24     model = GCNModel(hidden_dim=128, out_dim= 256, feature_dim =
ppi_network.shape[0])

```

3.2 训练GCN模型

在本部分的训练过程中，我们通过构建基于图卷积网络（GCN）的模型来对 PPI 网络中的节点进行嵌入学习。通过给已知致病基因标记为 1，并将其余节点标记为 0，我们能够利用有标签的数据进行监督学习，同时保证模型能够学习到基因间的潜在相互关系。

具体而言，我们采用了 Masked Cross-Entropy Loss 作为损失函数，确保模型仅在标注为 1 的致病基因上进行优化，从而避免对未标记基因节点的学习干扰。通过归一化邻接矩阵，使得每个节点的特征在图卷积过程中可以更加稳定地聚合其邻居节点的特征，从而更好地捕捉图结构信息。

在模型训练过程中，我们每一轮都通过 `TrainStep` 计算损失并进行参数更新，直至训练完成。训练得到的嵌入向量不仅为每个节点提供了一个低维特征表示，还能够用于后续的致病基因预测。

在标记过程中，已知的致病基因（在 `GeneSetPos` 中的基因）被标记为 1，这些基因作为正类，模型通过这些已标记的节点学习特征和图结构关系，从而为其他未标记节点的潜在致病性提供预测依据。

Masked Cross-Entropy Loss

MaskedSoftmaxCrossEntropy 类：该模块用于处理半监督节点分类场景下的损失函数计算。

- 输入：

- `logits` : 模型预测输出（维度 `[N, 2]`，即每个节点属于两个类别的概率）。
- `labels` : 标签数组，维度 `[N]`，仅部分节点有真实标签。
- `mask` : 布尔向量，标记哪些节点具有真实标签，维度 `[N]`。

- 核心逻辑：

- 使用 `nn.SoftmaxCrossEntropyWithLogits` 逐个节点计算损失。
- 将 `mask` 转换为 float 后与 `losses` 相乘，实现对未标注节点的屏蔽（其损失变为 0）。
- 最后将 masked 损失进行求和后，除以 mask 中非零的节点数，实现对“真实有标签节点”上的损失平均。

代码块

```

1  # 1) Masked Cross-Entropy Loss
2  class MaskedSoftmaxCrossEntropy(nn.Cell):
3      def __init__(self):
4          super().__init__()
5          self.ce = nn.SoftmaxCrossEntropyWithLogits(sparse=True,
reduction='none')
6          self.reduce_sum = ops.ReduceSum()
7          self.cast = ops.Cast()
8
9      def construct(self, logits, labels, mask):
10         # logits: [N, 2], labels: [N], mask: [N]
11         losses = self.ce(logits, labels)           # [N], 全部节点的 loss
12         mask_f = self.cast(mask, losses.dtype)     # 布尔 -> float
13         masked = losses * mask_f                   # 未标注位置被乘成 0
14         # 平均到“真正有标签”的节点上
15         return self.reduce_sum(masked) / self.reduce_sum(mask_f)

```

训练步骤封装

TrainStep：将一次前向传播、损失计算、反向传播和参数更新进行封装。

- 输入：
 - `A`：归一化后的邻接矩阵 `A_norm`。
 - `labels`：所有节点的标签向量。
 - `mask`：标签掩码，用于指定哪些节点可用于监督训练。
- 核心逻辑：
 - 封装 `forward()` 函数用于计算当前损失。
 - 使用 MindSpore 的 `ops.GradOperation` 计算网络参数的梯度。
 - 调用 `optimizer` 执行梯度更新。

训练循环中每个 epoch 会调用该类，实现结构清晰、逻辑分明的训练过程。

代码块

```

1  # 2) 训练步骤封装
2  class TrainStep(nn.Cell):

```

```

3     def __init__(self, network, loss_fn, optimizer):
4         super().__init__()
5         self.network = network
6         self.loss_fn = loss_fn
7         self.optimizer = optimizer
8         self.grad_op = ops.GradOperation(get_by_list=True)
9         self.params = network.trainable_params()
10
11     def construct(self, A, labels, mask):
12         def forward(A, labels, mask):
13             logits = self.network(A)
14             loss = self.loss_fn(logits, labels, mask)
15             return loss
16
17         grad_fn = self.grad_op(forward, self.params)
18         grads = grad_fn(A, labels, mask)
19         self.optimizer(grads)
20
21         # 返回当前 loss 值用于打印
22         return forward(A, labels, mask)

```

训练驱动逻辑

- **标签与掩码构建：**

- 初始化标签数组 `raw_labels`，将致病基因集合 `GeneSetPos` 对应位置设为 1（正类），其他为 0。
- 构建布尔向量 `mask`，仅对已知致病基因位置设为 `True`，用于 loss 屏蔽机制。

- **邻接矩阵归一化：**

- 使用对称归一化，保证 GCN 聚合邻居特征时稳定、均衡。

$$\hat{A} = \tilde{D}^{-1/2}(A + I)\tilde{D}^{-1/2}$$

- **实例化模型训练组件：**

- `loss_fn`：使用封装好的 `MaskedSoftmaxCrossEntropy`。
- `optimizer`：采用 `Adam` 优化器，学习率设为 `1e-4`。
- `train_step`：使用自定义 `TrainStep` 类组织完整训练过程。

- **启动训练循环：**

- 总共训练 `num_epochs = 10` 轮，每轮调用 `train_step`。
- 输出当前 epoch 的 loss 值，便于观察收敛情况。

```

1  # 3) 整体训练代码
2  # 假设你已经有 ppi_network, GeneSetPos
3  num_nodes = ppi_network.shape[0]
4  # 3.1 重新准备标签 & mask
5  raw_labels = np.zeros((num_nodes,), dtype=np.int32)
6  raw_labels[GeneSetPos] = 1
7  labels = Tensor(raw_labels)
8  mask = Tensor((raw_labels == 1))
9
10 # 3.2 归一化邻接
11 def normalize_adj(adj):
12     '''code here'''
13     ''''''
14
15 A_norm = Tensor(normalize_adj(ppi_network).astype(np.float32))
16
17 # 3.3 实例化模型、loss、optimizer、train_step
18 loss_fn = MaskedSoftmaxCrossEntropy()
19 optimizer = nn.Adam(model.trainable_params(), learning_rate=1e-4)
20 train_step = TrainStep(network=model, loss_fn=loss_fn, optimizer=optimizer)
21 print("Traning begin.")
22 # 3.4 训练循环
23 num_epochs = 10
24 for epoch in range(num_epochs):
25     loss = train_step(A_norm, labels, mask)
26     print(f"Epoch {epoch:03d}, loss={loss.asnumpy():.4f}")
27 print("Training completed.")

```

补充文献

这部分参考文献 Kipf 等人提出的 "**Semi-Supervised Classification with Graph Convolutional Networks**" (2016) 中的方法，该方法首次将图卷积网络（GCN）应用于半监督学习问题。在该文献中，Kipf 等人提出了一种图卷积层的设计，通过聚合每个节点及其邻居的特征信息，来学习节点的表示。GCN 在仅使用一小部分带标签的数据的情况下，通过高效地利用图结构信息，能够进行节点分类任务。

感兴趣的同学们可以看文献更深入地了解该方法。

文献: <https://arxiv.org/pdf/1609.02907>

4. 致病基因预测

在通过 GCN 模型学习到全图节点嵌入之后，我们希望根据已知致病基因集合的嵌入，通过计算相似度预测其他可能的关联基因。

余弦相似度的定义

余弦相似度 (Cosine Similarity) 是衡量两个向量夹角相似度的常用方法，定义如下：

$$\text{CosineSimilarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \cdot \|\mathbf{v}\|_2} = \frac{\sum_{i=1}^d u_i v_i}{\sqrt{\sum_{i=1}^d u_i^2} \cdot \sqrt{\sum_{i=1}^d v_i^2}}$$

- $\mathbf{u}, \mathbf{v}$ 为两个嵌入向量；
- 分子是内积，分母是两个向量的L2范数（模长）；
- 取值范围在 $[-1, 1]$ ，值越大表示越相似。

代码块

```
1  # 余弦距离
2  def cosine_similarity(vec1, vec2):
3      # 将输入转换为 NumPy 数组
4      vec1 = np.array(vec1)
5      vec2 = np.array(vec2)
6
7      # 检查向量长度是否一致
8      if len(vec1) != len(vec2):
9          raise ValueError("两个向量的维度必须相同！")
10
11     '''code here'''
12     # 计算向量点积
13     # 计算向量的模长 norm_vec1, norm_vec2
14     '''
15
16     # 避免除以零的情况
17     if norm_vec1 == 0 or norm_vec2 == 0:
18         raise ValueError("向量中存在零向量，无法计算余弦相似度！")
19
20     '''code here'''
21     # 计算余弦相似度 similarity
22     '''
23
24     return similarity
```

4.1 计算相似度并排序

该模块旨在利用图神经网络模型（GCN）获得每个基因在蛋白质相互作用网络（PPI）中的低维嵌入表示（embedding），并通过与已知致病基因集合嵌入的平均向量进行相似度计算，从而预测潜在致病基因。

嵌入提取：使用训练后的 GCN 模型 获取所有节点（基因）的嵌入向量表示，存储在 `embeddings_tensor` 中。通过 `.asnumpy()` 方法将其转为 NumPy 格式，维度为 `[num_nodes, embedding_dim]`。

致病基因集合中心表示构建：从嵌入中提取出已知致病基因集合 `GeneSetPos` 对应的向量子集，随后计算其按行的平均值，得到代表整个致病基因集合的“中心嵌入向量”。

嵌入相似度计算：遍历所有非致病基因，对于每一个非致病基因 获取其嵌入向量，并与所有致病基因嵌入向量的平均值 `GeneSetEmbedding` 计算余弦相似度。将所有相似度结果保存在列表中，并记录对应基因索引于 `gene_indices` 中。

相似度排序与候选基因筛选：将相似度和对应基因编号组合，并按相似度从高到低排序。相似度越高，说明该基因越可能与致病基因功能相关，越可能为潜在致病基因。

代码块

```
1  # 获取节点嵌入
2  embeddings_tensor = model(A_norm)
3  embeddings = embeddings_tensor.asnumpy()
4
5  GeneSetPos_embeddings = embeddings[GeneSetPos] # 提取对应下标的嵌入
6  GeneSetEmbedding = np.mean(GeneSetPos_embeddings, axis=0) # 按行求平均
7  print(GeneSetEmbedding)
8
9  # 初始化存储结果的列表
10 distances = []
11 gene_indices = []
12 # 遍历所有基因（排除致病基因集合中的基因）
13 for i in range(embeddings.shape[0]):
14     if i not in GeneSetPos: # 排除致病基因集合中的基因
15         # 计算当前基因与致病基因集合的余弦距离
16         # 余弦距离
17         distance = cosine_similarity(embeddings[i], GeneSetEmbedding)
18         distances.append(distance)
19         gene_indices.append(i)
20
21 # 将距离和基因下标组合成一个数组
22 distance_gene_pairs = list(zip(distances, gene_indices))
23
24 # 按余弦相似度从高到低排序（相似度越高，越靠前）
25 distance_gene_pairs.sort(key=lambda x: x[0], reverse=True)
```

4.2 输出排序后的top 10基因

这部分完成嵌入和相似度计算后，我们从所有候选基因中**选出相似度最高的前 k 个基因**，作为预测的重点关注对象。

代码块

```
1  # 输出 Top-k 个最相似的基因
2  k = 10 # 设置 Top-k 的值
```

```
3 top_k_genes = distance_gene_pairs[:k]
4
5 print(f"选择的疾病ID为: {DiseaseID},其对应描述为:{getDiseaseDescription(DiseaseID)
  [0][0]}")
6 # 打印结果
7 print(f"Top-{k} 最可能的致病基因为:")
8 for distance, gene_idx in top_k_genes:
9     print(f"基因 {gene_idx}: {gene_name[gene_idx][0][0]} (余弦距离:
    {distance:.4f})")
```

返回结果如下:

选择的疾病ID为: 114480,其对应描述为:BREAST CANCER

Top-10 最可能的致病基因为:

基因 125: STAT3 (余弦距离: 0.9669)

基因 163: EP300 (余弦距离: 0.9643)

基因 154: AR (余弦距离: 0.9636)

基因 144: JUN (余弦距离: 0.9631)

基因 157: STAT1 (余弦距离: 0.9617)

基因 1759: CEBPB (余弦距离: 0.9603)

基因 71: PML (余弦距离: 0.9594)

基因 1379: MYC (余弦距离: 0.9584)

基因 85: DAXX (余弦距离: 0.9578)

基因 237: SP1 (余弦距离: 0.9578)